

Procesamiento de Lenguaje Natural con Aprendizaje Profundo, Máster en Ciencia y Tecnología Informática Curso 2025-2026

Tema 5 Transformers

¿Por qué este tema?

Los modelos actuales de IA (GPT-4/5, Gemini, Claude, LLaMA, etc.) están basados en Transformers.

- self-attention
- multi-head attention



Entender atención = entender cómo funcionan los LLMs

Índice

- Repaso Arquitecturas Profundas para PLN y modelos de word embeddings.
- Mecanismo de atención
- Arquitectura Transformer
 - Primeros transformers: BERT, GPT, T5
 - Modelos actuales (LLMs).
- Transfer Learning

Enfoques de Deep Learning para PLN: repaso

- Arquitecturas profundas:
 - CNN (tema 2)
 - RNN (LSTM y GRU) (tema 3)
- Representación de palabras
 - Word Embeddings (tema 4)

¿Cómo procesa una CNN un texto?

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	
gusta+la+tortilla	
la+tortilla+de	
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	
la+tortilla+de	
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	-3,6
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	-3,6
tortilla+de+patatas	-0,2
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	-3,6
tortilla+de+patatas	-0,2
de+patatas+pad	2.0

CNN

- Pipeline básico de una CNN:

Embeddings → Convolución → Pooling → (opcional) Dense → Capa de salida

- Capa de salida:
 - a. Clasificación binaria: Sigmoid (probabilidad de la clase positiva)
 - b. Multi-clasificación: Softmax (probabilidades de cada clase)
 - c. Multilabel: sigmoid (probabilidad para cada etiqueta)
 - d. Regresión: lineal ($y=Wh+b$)

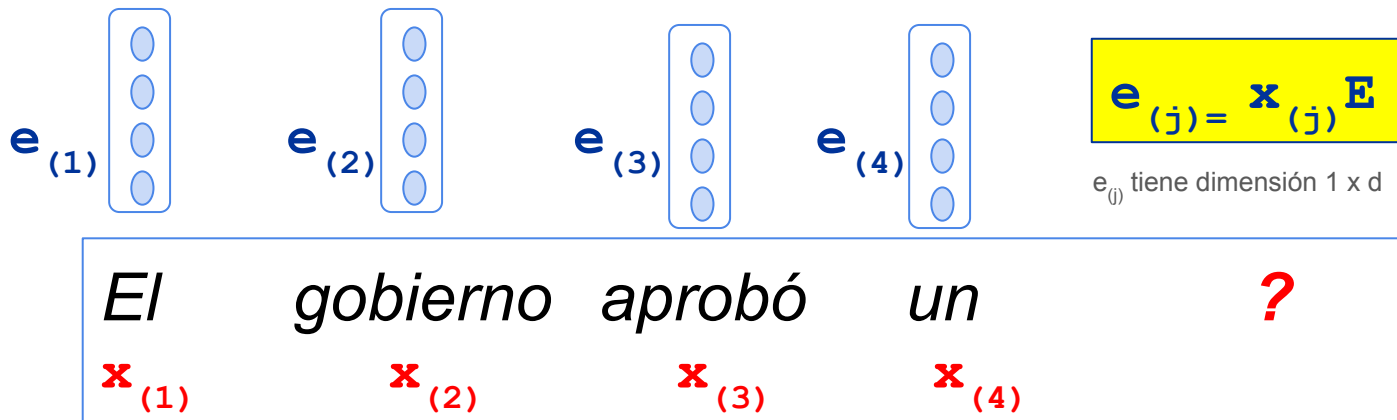
Ventajas y Limitaciones de las CNNs

- Ventajas:
 - Captura patrones locales (n-gramas) en el texto
 - Eficientes y paralelizables
 - Buenos resultados en clasificación de texto.
- Limitaciones:
 - No pueden capturar dependencias largas (entre palabras distantes)
 - El orden de las palabras solo se captura en patrones locales, no en todo el texto.

¿Cómo procesa una RNN un texto?

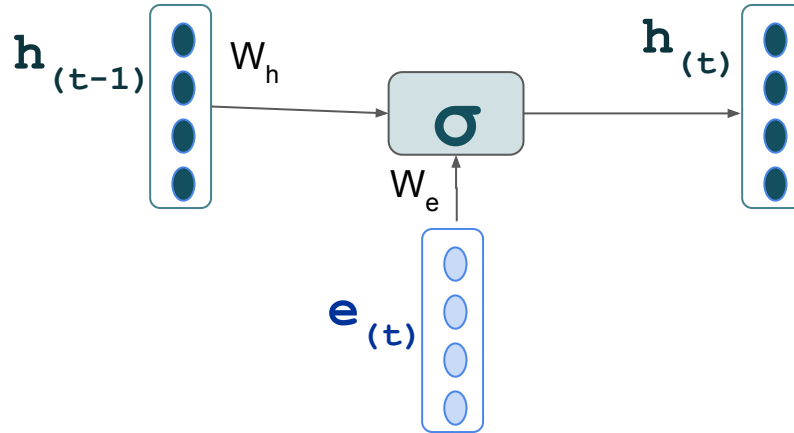
Redes recurrentes (RNN)

- Cada paso temporal (token) se representa con un **embedding**.
- La capa de entrada está formada por los embeddings de la secuencia.



Redes recurrentes (RNN) - capa recurrente

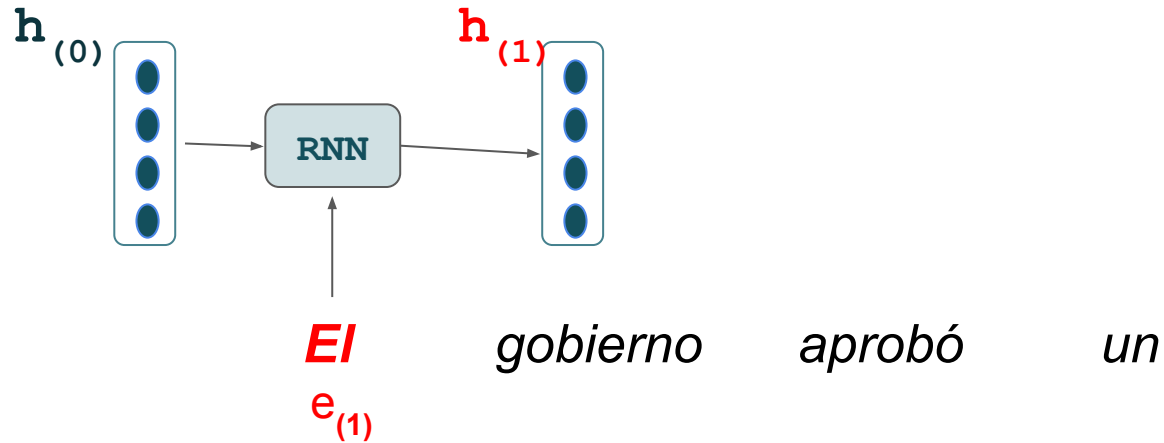
En una **RNN vanilla** (versión más básica), **función de activación: tangente hiperbólica (tanh), o sigmoide.**



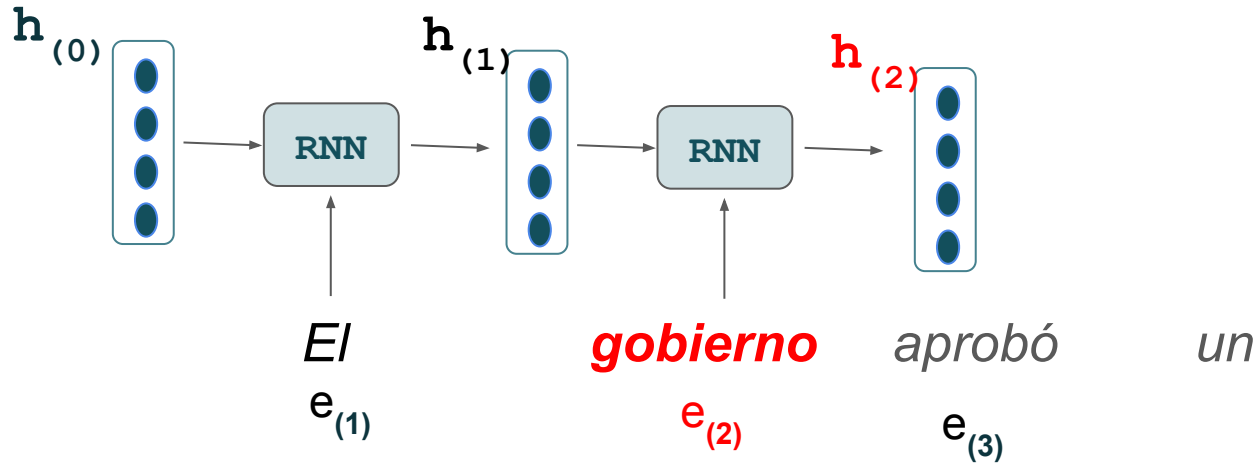
$e_{(j)}$ vector dimensión d

- **LSTM:** introduce **gates y memoria** para capturar dependencias largas.
- **GRU:** **simplifica LSTM** usando menos puertas y menos parámetros.

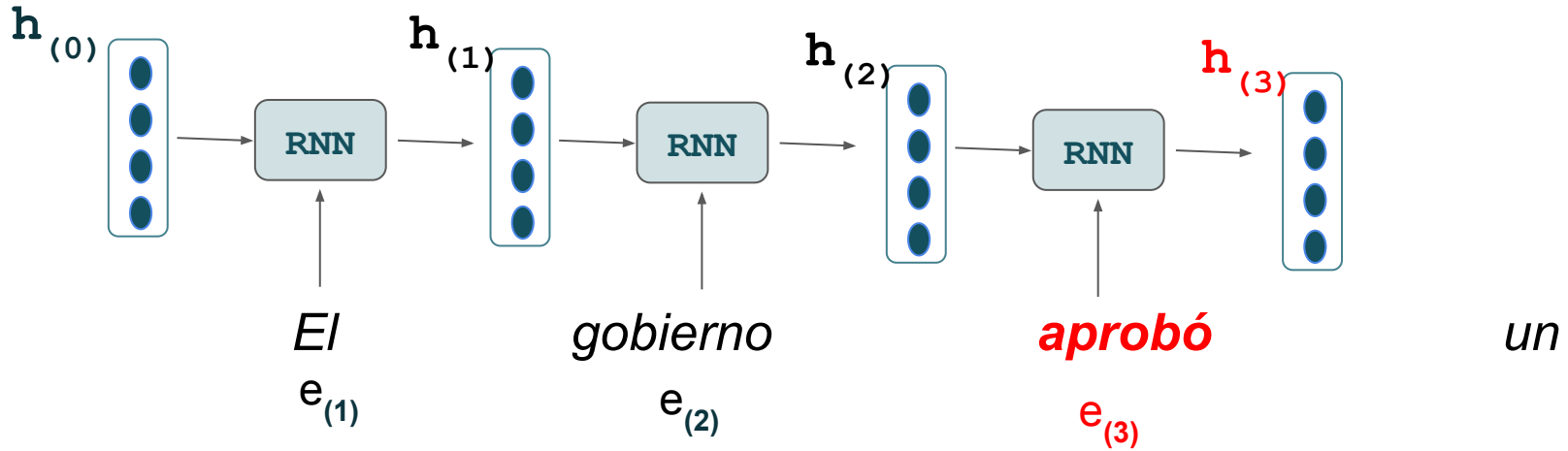
Redes recurrentes (RNN)



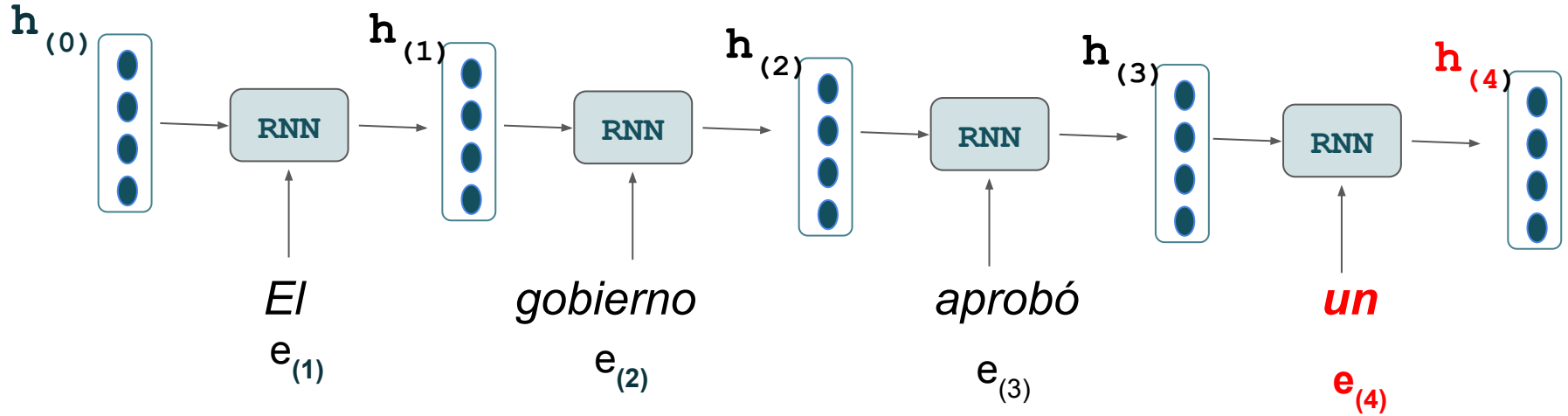
Redes recurrentes (RNN)



Redes recurrentes (RNN)



Redes recurrentes (RNN)



Capa de salida en RNN

- Pipeline básico de una RNN:

Embeddings → RNN (LSTM / GRU) → (opcional) Dense → Capa de salida

- Capa de salida:
 - Modelo de Lenguaje: Softmax (probabilidad sobre el vocabulario)
 - Clasificación: Sigmoid / Softmax
 - Multilabel: sigmoide (probabilidad para cada etiqueta)
 - Regresión: lineal ($y=Wh+b$)
 - Etiquetado secuencial (NER o Análisis morfosintáctico): Softmax o CRF.

Ventajas y Limitaciones de las RNN

Ventajas:

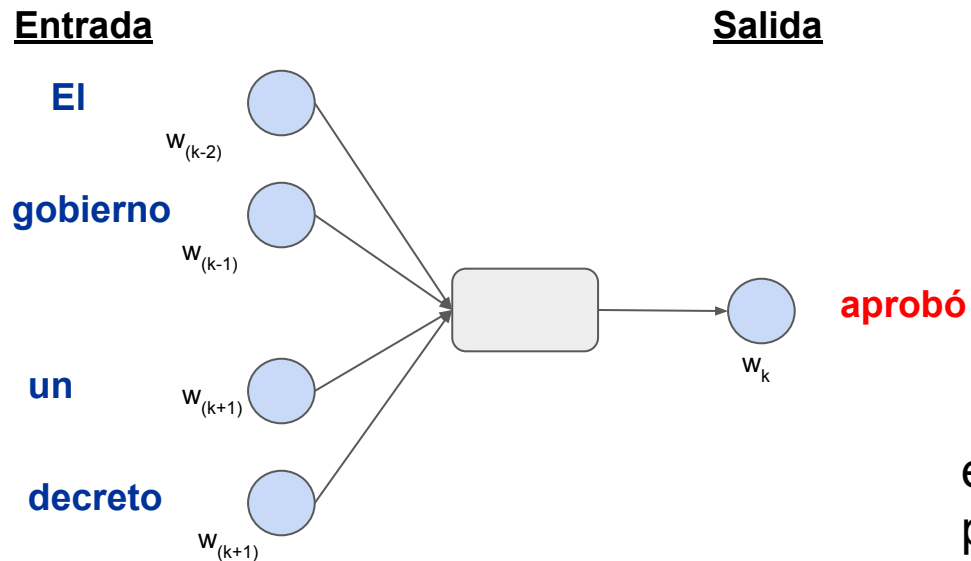
- Tienen en cuenta el orden del texto
- Capturan relaciones entre palabras
- Buenos resultados en tareas secuenciales (modelado de lenguaje, análisis morfosintáctico, reconocimiento de entidades).

Limitaciones:

- No son paralelizables (procesamiento secuencial).
- Dificultad para capturar dependencias muy largas

**¿Cómo aprenden los modelos de word embeddings
los vectores de palabras?**

Word2Vec: CBOW



el objetivo de la red es predecir una palabra a partir de su contexto

window: size = 5

Ventajas y Limitaciones de los modelos Word Embeddings

Ventajas:

- Vectores densos de palabras
- Capturan relaciones semánticas:
 - Similitud entre palabras
 - Relaciones semánticas (p. ej., *perro–animal*, *rey–reina*, *Madrid–España*, *enseñar–profesor*)
- Captura relaciones morfosintácticas (p. ej: *cat-cats*, *running-run*, *he-his*; *slow-slower-slowest*).
- Mejoran el rendimiento de modelos neuronales

Principal limitación: dada palabra tiene un único vector (independientemente de su contexto)

- *El mono se comió un plátano* (animal)
- *Qué mono es este peluche!* (bonito)
- *El mecánico llevaba un mono azul* (prenda de vestir)
- *Tengo un mono de nicotina* (síndrome de abstinencia)

¿Cómo capturar **dependencias largas**?

¿Cómo representar **palabras polisémicas**?

¿Cómo **procesar el texto en paralelo** de forma eficiente?

Mecanismo de atención

Cuando interpretamos una palabra, no todas las palabras son igual de importantes.

The animal didn't cross the street because it was too tired.

¿A qué palabra se refiere *it*?: animal o street?

The animal didn't cross the street because it was too tired



Intuición: El modelo debe **identificar qué palabras del contexto son más relevantes**.

Self-Attention

En self-attention, cada palabra “mira” a las demás y decide cuáles son importantes (construye su representación usando el contexto)

Para cada palabra:

- Se compara su representación con las representaciones de las demás palabras.
- Se calcula un peso de atención para cada palabra.
- Se obtiene una representación contextualizada.

Cómo se calcula la atención?

Para cada palabra, el modelo crea **tres representaciones vectoriales**:

- **Query (Q)** → representa **qué información necesita esta palabra**.

Por ejemplo, para *it*, el modelo quiere encontrar a qué entidad se refiere.

- **Key (K)** → representa **qué tipo de información tiene cada palabra**.

Por ejemplo, *animal* es una entidad animada, mientras *street* es un lugar.

- **Value (V)** → información que se **combina** para construir la representación final.

Si **animal** recibe mucha atención, su **value** se usará más para construir la representación final de **it**.

Cómo se calcula la atención?

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V$$

- $x \in \mathbb{R}^{d_{model}}$
- $Q, K \in \mathbb{R}^{d_k}$
- $V \in \mathbb{R}^{d_v}$

$$W_Q \in \mathbb{R}^{d_{model} \times d_k}$$

$$W_K \in \mathbb{R}^{d_{model} \times d_k}$$

$$W_V \in \mathbb{R}^{d_{model} \times d_v}$$

W_Q, W_K, W_V son matrices de pesos (entrenamiento).

Valores muy usados:

- $d_{model}=512$, ($d_{model}=768$ Bert-base, $d_{model}=1024$ Bert-large)

En multi-head attention se suele hacer:

$$d_k = d_v = \frac{d_{model}}{h}$$

donde h es el número de heads.

Cómo se calcula la atención?

Pasos:

$$Q = xW_Q$$

1. Cálculo de Q, K, V

$$K = xW_K$$

$$V = xW_V$$

2. Cada **Query** se compara con todas las **Keys** usando producto escalar: $score_{ij} = Q_i \cdot K_j$

3. Convierte el score en peso de atención: $\alpha_{ij} = softmax\left(\frac{Q_i K_j}{\sqrt{d_k}}\right)$

4. Los pesos se usan para combinar los **Values**

$$\sum_j \alpha_{ij} V_j$$

Es la **representación final** de la palabra i

Ventajas de self-attention

- Usa todo el contexto de la oración
- Captura dependencias largas
- Permite procesamiento en paralelo (porque no requiere procesar las palabras secuencialmente)

Limitación de una sola atención

Con una sola atención, el modelo aprende **un único patrón de atención**.

Pero en una oración pueden existir distintos tipos de relaciones entre palabras.

Ejemplo: *The animal didn't cross the street because it was too tired*

Posibles relaciones:

- Correferencia: *it* → animal (pronombre - anáfora)
- Relación semántica: *tired* → animal (dependencia larga)
- Relación sintáctica: *cross* → street (verbo-objeto)

Solución: **Multi-head attention**

Multi-head attention

Para capturar **distintos tipos de relaciones entre palabras**, el modelo utiliza **varios mecanismos de atención en paralelo** (*attention heads*).

Cada *head* aprende **un patrón de atención diferente**.

1. Cada *head* calcula su **propia atención**.
2. Los resultados se **concatenan** y se proyectan nuevamente.

$$head_i = Attention(Q_i, K_i, V_i)$$

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W_O$$

¿Qué aprende cada cabeza?

- Relaciones sintácticas (dependencias)
- Relaciones semánticas (similitud de significado)
- Coreferencias (él/ella/esto)
- Negación y modificadores

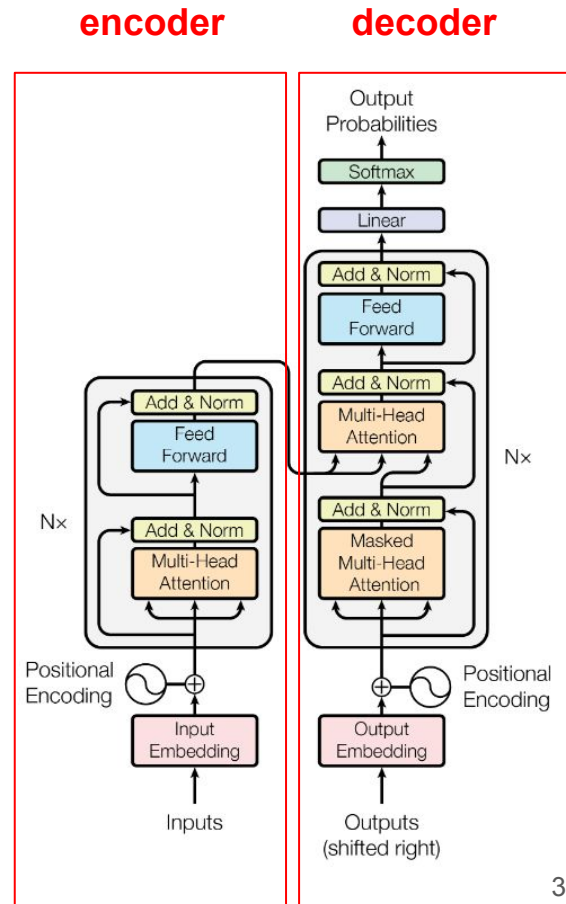
Visualización de capas de atención

El siguiente [notebook](#) contiene código para mostrar las capas de atención.

Primer modelo transformer

- Vaswani, A. et al. (2017). **Attention is all you need.** *Advances in neural information processing systems*, 30.
- Basada en self-attention y **multi-head**
- Propuesto para Machine Translation
- Encoder (6 bloques) y Decoder (6 bloques)
- Cada bloque se componen:

**Multi-head self-attention -> Feed-forward
-> Normalization**



Principales familias de modelos Transformers

Modelos Encoders: comprenden texto (BERT).

Modelos Decoder: generación de texto (GPT)

Modelos Encoder–decoder: text to text (T5).

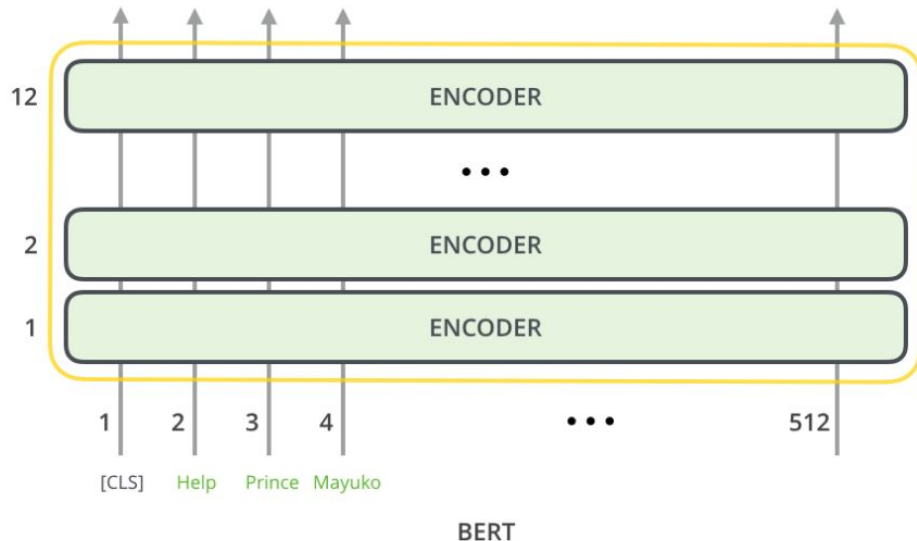
BERT: Bidirectional Encoder Representations from Transformers

Pila de encoders:

BERT-base = 12 encoders,
embedding size= 768

BERT-large = 24 encoders,
embedding size = 1024

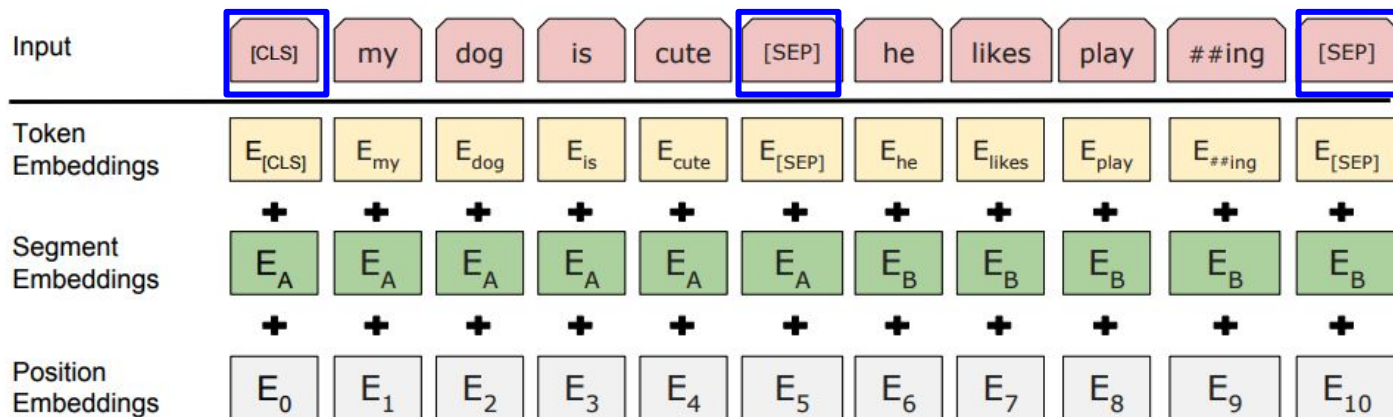
Tamaño máximo: 512 tokens



Pre-entrenado a partir de textos de Wikipedia (2,500 millones de tokens) y BookCorpus (más de 800 millones de tokens)

Formato de la entrada de BERT

Token embedding+Segment embedding+Position embedding



Token embedding: representación de cada token

Segment embeddings: Indican a qué oración pertenece cada token.

Positional embeddings: Codifican la posición del token en la secuencia.

Pretraining de BERT

- Se aplican a la vez:
 - Masked Language Model (MLM)
 - Next Sentence Prediction (NSP)

Masked Language Modeling

- Durante el pretraining:
 - se selecciona aleatoriamente algunos tokens y se reemplazan por [MASK].
 - El modelo debe predecir el token original.

Oración	Entrada	Objetivo
El gato está en el sofa	El <MASK> está en el sofa	gato
Madrid es la capital de España	Madrid es la <MASK> de España	capital

Next Sentence Prediction (NSP)

BERT recibe **pares de oraciones** y debe predecir si la segunda oración **sigue realmente a la primera en el texto.**

Entrada

[CLS] The animal didn't cross the street. [SEP] It was too tired. [SEP]

Objetivo: Clasificar el par de oraciones como:

- **IsNextSentence** → la segunda oración sigue a la primera
- **NotNextSentence** → la segunda oración es aleatoria

Modelos basados en BERT

- RoBERTa: BERT entrenada con más datos. No usa NSP.
- ALBERT: BERT reduce parámetros compartiendo pesos entre capas.
- DistilBERT: BERT más pequeña (6 capas).
- Modelos pre-entrenados con textos en **español**: BETO, Maria.
- Modelo **multilingües**: mBERT, xlm-RoBERTa
- Modelos entrenados con textos de **redes sociales**: BERTweet, Robertuito,
- Modelos **específicos** de dominio: BioBERT (biomedicina), ClinicalBERT (clínico), SciBERT (ciencia), FinBERT (finanzas), LegalBERT (derecho).

GPT (Generative Pretrained Transformer)

Transformer para generar texto. Solo decoder.

El modelo aprende a predecir la siguiente palabra dado el contexto anterior.

Entrada: *The animal didn't cross the street because*

Salida: *it -> was -> too -> tired*

GPT → GPT-2 → GPT-3 → GPT-4 -> ... -> GPT 5.4

T5 (Text-to-Text Transfer Transformer)

Modelo encoder–decoder (Transformer completo)

Pretraining: denoising

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.

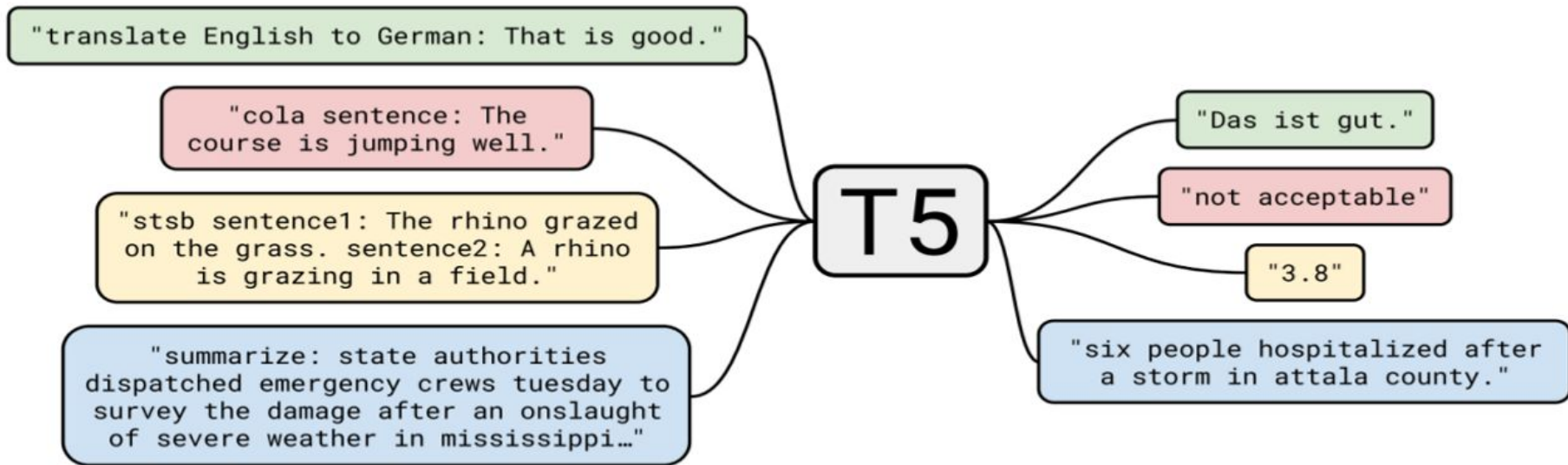
Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

T5 (Text-to-Text Transfer Transformer)



LLMs (GPT 5.4, Gemini, Claude, LLaMA...)

Transformers entrenados a gran escala.

-  Mucho más datos (billones de tokens)
-  Mucho más grandes (miles de millones de parámetros)
-  Mejor entrenamiento (instruction tuning, aprendizaje por refuerzo)
- 👉 Prompting (instrucciones en lenguaje natural)
- ✓ Resolución de tareas complejas

Resumen: Evolución de los modelos de lenguaje

1. Modelos clásicos (BoW, TF-IDF).

⚠ No capturan orden ni contexto

2. Word Embeddings (Word2Vec, Glove).

⚠ Una palabra = un único vector (sin contexto)

3. Deep Learning para PLN (CNN, RNN).

⚠ Problemas con dependencias largas y eficiencia

4. Transformers (BERT, GPT, T5)

✓ Paralelización. Capturan dependencias largas

5. Modelos actuales (GPT-4/5, Gemini, Claude, LLaMA).

✓ Capaces de generar, entender y razonar sobre lenguaje

¿Qué es transfer learning?

Transfer learning

👉 Aprovechar un modelo ya entrenado para una nueva tarea. ¿Cómo?

1. Pretraining: Modelo entrenado con grandes cantidades de texto. Aprende lenguaje general
2. Fine-tuning: Se adapta a una tarea específica (clasificación, NER, QA, etc.)

¿Qué es pretraining?

Pretraining

- Entrenamiento masivo sobre grandes corpus no etiquetados
- No está especializado en una tarea concreta
- Aprende representaciones generales del lenguaje
- ¿Cómo se entrena?
 - Masked Language Modeling (BERT)
 - Causal Language Modeling (GPT)

¿Qué enseña realmente el pretraining?

Aprende una representación general del lenguaje

No se entrena explícitamente para:

- Clasificar
- Traducir
- Resumir
- Responder preguntas

¿Qué es fine-tuning?,
¿Qué es instruction-tuning?
¿Qué es prompting?

Fine-Tuning

👉 Adaptar el modelo a una tarea específica

- Ejemplo:
- Input: "I love this movie"
- Output: "positive"

- ✓ Aprende una tarea concreta
- ✓ Dataset etiquetado específico

Instruction Fine-Tuning

-  Entrena el modelo con instrucciones
- Ejemplo:
 - Input: "Clasifica el sentimiento: I love this movie"
 - Output: "positive"
-  Aprende a interpretar instrucciones
-  Puede hacer muchas tareas distintas

Prompting

-  Usar el modelo mediante instrucciones en lenguaje natural.
- No cambiamos el modelo.
- No requiere entrenamiento adicional. Solo cambiamos la entrada (prompt)
- Ejemplo:
- Input: "Clasifica el sentimiento: I love this movie"
- Output: "positive"

Caso práctico: análisis de sentimiento

Desarrolla tres notebooks independientes que resuelvan el problema de clasificación de sentimiento sobre el dataset [imdb](#), usando:

- Fine-tuning
- Instruction-fine tuning
- Prompting.

⚠ Para reducir el tiempo de ejecución, trabaja con una muestra pequeña del dataset (por ejemplo, entre 1.000 y 5.000 ejemplos).

⚠ Evalúa también sobre una muestra reducida del conjunto de test. Asegurate que los tres enfoques se entrena y evalúan con los mismos datos.

Caso práctico: análisis de sentimiento



Análisis. Compara los tres enfoques y responde:

- ¿Qué diferencias hay en el proceso de entrenamiento?
- ¿Cuál es más flexible?
- ¿Cuál requiere más datos?
- ¿Cómo influye el diseño del prompt en los resultados?
- ¿Qué ventajas e inconvenientes presenta cada enfoque?